

Git, GitHub y Flujo de Trabajo

Diego Muñoz

19 de junio de 2026

Objetivos de la Clase

- Entender qué es el control de versiones y por qué lo necesitas.
- Diferenciar Git de GitHub.
- Ordenar tu área de trabajo para no perder tu progreso.
- Manejar el flujo básico: add, commit, push.
- Entregar tus tareas como repositorios.

El problema

Cuando guardas copias del proyecto para no perder lo anterior, terminas con archivos imposibles de seguir.

Ejemplo:

- tarea.js
- tarea_v2.js
- tarea_final.js
- tarea_final_definitivo.js

No hay forma de saber cuál es la buena ni qué cambió entre una y otra.

La idea

Un sistema de control de versiones guarda el historial completo de tu proyecto y te permite volver a cualquier punto anterior.

Ejemplo: No es guardar `tarea_v2` y `tarea_final` por separado. Es una sola carpeta con un historial: cada commit es una “foto” a la que puedes volver.

Definición

Git es un programa que corre en tu computador y administra el historial de cambios de un proyecto.

- Guarda cada cambio que confirmas.
- Te deja comparar, volver atrás y trabajar en paralelo.
- Es el estándar que vas a encontrar en cualquier empresa.

Git no es GitHub

- **Git:** el programa local que registra los cambios.
- **GitHub:** un servicio en internet donde subes y compartes tus repositorios.

Ejemplo: Git es tu cuaderno de borradores. GitHub es la nube donde lo respaldas y lo entregas. Hay otros como GitLab o Bitbucket.

Identifícate una vez

Antes de tu primer commit, Git necesita saber quién eres. Se configura una sola vez por computador y queda registrado en cada commit.

```
git config --global user.name "Tu Nombre"
```

```
git config --global user.email "tu@correo.com"
```

Una regla simple

Un proyecto vive en una sola carpeta, y esa carpeta es un repositorio. No anides un repositorio dentro de otro y evita los espacios en los nombres.

Ejemplo:

- Bien: tarea-01-landing
- Mal: Tarea Final (1)

git init

Creas la carpeta del proyecto y la conviertes en repositorio. Recién ahí Git empieza a seguir tus cambios.

```
mkdir mi-proyecto
```

```
cd mi-proyecto
```

```
git init
```

- Sin `git init`, Git responde fatal: not a git repository.
- `git init` crea la carpeta oculta `.git/`: eso la hace un repo. **NO TOCAR!**

Qué dejar fuera

Versionas tu código fuente. No versionas lo que se regenera, lo que es enorme o lo que es **secreto**.

```
# .gitignore
node_modules/
dist/
.env  # <--- subir este archivo le costó su trabajo a más de un dev
```

Ejemplo: node_modules/ pesa cientos de MB y se reconstruye con un comando. No tiene sentido versionarlo.

Tres áreas

Tu trabajo pasa por tres etapas antes de quedar guardado en el historial.

- **Working directory:** los archivos que estás editando.
- **Staging:** lo que marcas para el próximo commit.
- **Repositorio:** los cambios ya confirmados.

En comandos

```
git status          # ver qué cambió  
git add archivo.js  # preparar un cambio  
git commit -m "Agrega formulario de login"
```

Ejemplo: Editaste tres archivos, pero solo terminaste uno. Agregas y haces commit. Los otros quedan para después.

Cuidado con `git add .`

`git add .` prepara de una vez todos los cambios pendientes. Se puede usar, pero hacerlo por costumbre y a ciegas es la raíz de problemas grandes.

- Antes de agregar, revisa con `git status` qué va a entrar.
- Úsalo sabiendo exactamente qué estás agregando.

Ejemplo: Un `git add .` sin mirar sube `node_modules/`, un `.env` con claves o un archivo a medio terminar.

Antes de conectar

El repositorio remoto tiene que existir y ser tuyo. Lo creas vacío en tu cuenta de GitHub, y necesitas estar autenticado para escribir en él.

- Crea el repo en GitHub primero, en tu cuenta.
- Git pide autenticación: solo subes a repos donde tienes permiso.

Repositorios remotos y GitHub

Subir tu trabajo

El repositorio remoto es la copia que vive en GitHub. Conectas tu repo local una vez y desde ahí envías tus commits.

HTTPS: lo más simple para empezar

```
git remote add origin https://github.com/usuario/repo.git
```

SSH: requiere configurar una llave una vez

```
git remote add origin git@github.com:usuario/repo.git
```

```
git push -u origin main
```

Ejemplo: Puedes hacer `git remote add origin`

`https://github.com/torvalds/linux.git`, pero el push falla: ese repo no es tuyo.

Traer y enviar

- `git clone`: descarga un repositorio completo a tu máquina.
- `git pull`: trae los cambios nuevos del remoto.
- `git push`: envía tus commits.

Ejemplo: Trabajas en el laboratorio y haces `push`. Llegas a casa, haces `pull` y sigues donde quedaste.

- Agregar algo al `.gitignore` tarde: si ya hiciste commit, sigue versionado. `.gitignore` solo ignora lo que aún no rastreas.
- Commits enormes con mensajes como “cambios” o “asdf”.
- `push` rechazado: el remoto tiene algo que no tienes. Haz `git pull` primero.
- Iniciar un repositorio dentro de otro.
- Creer que guardar el archivo ya es un commit.

Buenas prácticas de commits

- Pequeños y frecuentes, cada uno con un propósito.
- Mensaje en imperativo y claro: “Agrega login”, no “cambios”.
- Mejor un commit que funciona que uno enorme y roto.

La portada de tu repo

El `README.md` es el archivo que GitHub muestra al abrir el repositorio. Explica qué es el proyecto y cómo usarlo.

- `.md` es Markdown: texto plano con formato simple, como títulos, listas y enlaces.
- Va en mayúsculas por convención, para que se note y sea lo primero que se lee.

Ejemplo: Es lo primero que ve quien llega a tu repo. Sin README, es una caja sin etiqueta.

La entrega es un repositorio

Qué se espera

- Un README.md que explique qué es y cómo ejecutarlo.
- Estructura de carpetas ordenada.
- Historial de commits que muestre tu avance.
- El enlace al repositorio en GitHub.

Ejemplo: Entregas un link a GitHub, no un .zip por correo.

Comandos esenciales

```
git init           # crear repo local
git status         # estado actual
git add .          # preparar todo
git commit -m ""   # confirmar cambios
git push           # subir al remoto
git pull           # actualizar desde el remoto
git log --oneline  # ver historial
```

1. Git versiona tu trabajo. GitHub lo aloja y lo comparte.
2. Un proyecto, una carpeta, un repositorio. Usa `.gitignore`.
3. Flujo: editar, add, commit, push.
4. Tus tareas se entregan como repositorio.

- [Documentación oficial de Git](#)
- [Guías de GitHub](#)
- [Pro Git, libro gratuito](#)

¿Tienes dudas? ¡Hablemos!